JSON & Web APIs

Ambient intelligence course (A.Y. 2025/2026)
Topic taught by Prof. Davide Loconte

Gabriele Colapinto

May 3, 2026

License

Notes from Ambient intelligence (A.Y. 2025/2026) by Gabriele Colapinto. Course taught by professors Michele Ruta, Floriano Scioscia, Arnaldo Tomasino, Davide Loconte — Licensed under CC BY-NC 4.0.
https://creativecommons.org/licenses/by-nc/4.0/

© 2025 Gabriele Colapinto
Released under the Creative Commons Attribution–NonCommercial 4.0 International License

Contents

1 Fundamentals of Data Exchange: JavaScript Object Notation (JSON) 5
1.1 JSON serialization, field order and unsupported values . . . 7
1.2 JSON Schema . . . 8
1.2.1 Purpose and Motivation . . . 8
1.2.2 Basic Structure . . . 9
1.2.3 Common Keywords . . . 9
1.2.4 Example with Validation Constraints . . . 10
1.2.5 Advantages . . . 10
1.3 JSON vs XML: Structure, Validation, and Performance . . . 10
1.3.1 Structural Differences . . . 10
1.3.2 Example and Space Comparison . . . 11
1.3.3 Validation Mechanisms . . . 11
1.3.4 Summary of Differences . . . 12

2 Web API 13
2.1 The Browser Environment: BOM and DOM Architectures . . . 13
2.1.1 The Browser Object Model (BOM) . . . 13
2.1.2 The Document Object Model (DOM) . . . 15
2.2 The Document Object: Attributes and Methods . . . 15
2.2.1 Document Attributes . . . 16
2.2.2 Document Methods . . . 16
2.2.3 Event Handlers . . . 17
2.2.4 Example of DOM manipulation and event handling . . . 17
2.3 The Element Interface: Attributes and Methods . . . 18
2.3.1 Element attributes . . . 18
2.3.2 Element methods . . . 19

3 Optimized Resource Execution: Script Loading Strategies 21

4 Other key Web APIs 23
4.1 Client-Side Persistence: Web Storage vs. Cookies . . . 23
4.1.1 Comparison: Web Storage vs. Cookies . . . 23
4.2 Geolocation API . . . 23

5 Asynchronous JavaScript (AJAX) and Network Interaction 25
5.1 Synchronous requests vs asynchronous requests . . . 25
5.1.1 Race Conditions in Search Inputs . . . 26
5.1.2 SEO and User Experience . . . 28
5.2 AJAX endpoints as attack surfaces and security considerations . . . 28
5.2.1 AJAX endpoints as vulnerability points . . . 28
5.2.2 Same-Origin Policy and CORS . . . 28
5.2.3 Defense Mechanisms . . . 29
5.2.4 Example of a Complete CORS Header Configuration . . . 29

5.3 The `XMLHttpRequest` (XHR) and the Fetch API 30
5.3.1 XHR Lifecycle and readyState property 30
5.3.2 XHR Methods and Execution Flow 31
5.3.3 XHR properties 32
5.3.4 Example of XMLHttpRequest 34
5.3.5 Fetch API: A Modern Approach 35
5.3.6 Example of request using the fetch API 35
5.3.7 XHR vs Fetch: Key Differences 36

1 Fundamentals of Data Exchange: JavaScript Object Notation (JSON)

Data interchange in modern web engineering is governed by the **JavaScript Object Notation (JSON)**, as defined under the **RFC 7159** standard. JSON serves as a language-independent, lightweight data format that enables communication across virtually all modern programming environments. Technically, the format uses the MIME type `application/json` and the file extension `.json`. Its strategic importance stems from being a proper subset of JavaScript syntax; since every JavaScript object is essentially an associative array, JSON leverages the language's native mechanism for handling keyed data, making it the most prominent exchange format in contemporary web architecture.

For example, let us consider the following JSON object:

```
let myJSONObject = {
        "bindings": [
                {"ircEvent": "PRIVMSG", "method": "newURI"},
                {"ircEvent": "PRIVMSG", "method": "deleteURI"},
        ]
};
```

We can traverse it and access its properties as follows:

```
myJSONObject.bindings[0].method // "newURI"
```

The structural integrity of JSON is derived from two primary components supported by nearly all modern programming languages (see figures 1.1 and 1.2):

Component	**Definition**	**Syntax**	**Use Case**
Object	An unordered set of name-value pairs.	Braces `{ }`	Representing records, dictionaries, or hash tables where keys map to values.
Array	An ordered list of values.	Brackets `[ ]`	Maintaining sequences, vectors, or lists where element order is critical.

JSON supports the following data types (see figure 1.3):

- **string** (bounded by double quotes)
- **number**
- **object**
- **array**

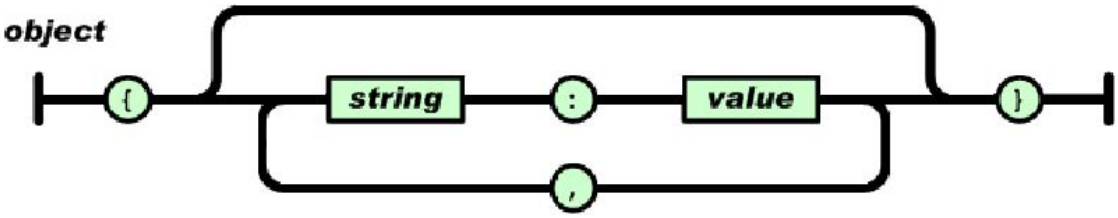

Figure 1.1: Object structure

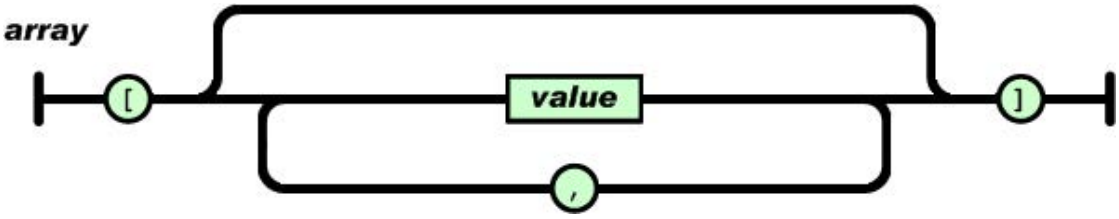

Figure 1.2: Array structure

- **boolean** (true/false)
- **null**

To transmit these structures, developers must utilize **serialization** via the `JSON.stringify` method to convert objects into strings.

```
let myJSONText = JSON.stringify(myObject);
```

Upon receipt, **deserialization** via `JSON.parse` is necessary to reconstruct the functional object.

```
let myObject = JSON.parse(myJSONtext);
```

The following nested JSON structure illustrates the representation of hierarchical data:

```
{
  "glossary": {
    "title": "example glossary",
    "GlossDiv": {
      "title": "S",
      "GlossList": {
        "GlossEntry": {
          "ID": "SGML",
          "SortAs": "SGML",
          "GlossTerm": "Standard Generalized Markup Language",
          "Acronym": "SGML",
          "Abbrev": "ISO 8879:1986",
          "GlossDef": {
            "para": "A meta-markup language, used to create markup
            languages such as DocBook.",
```

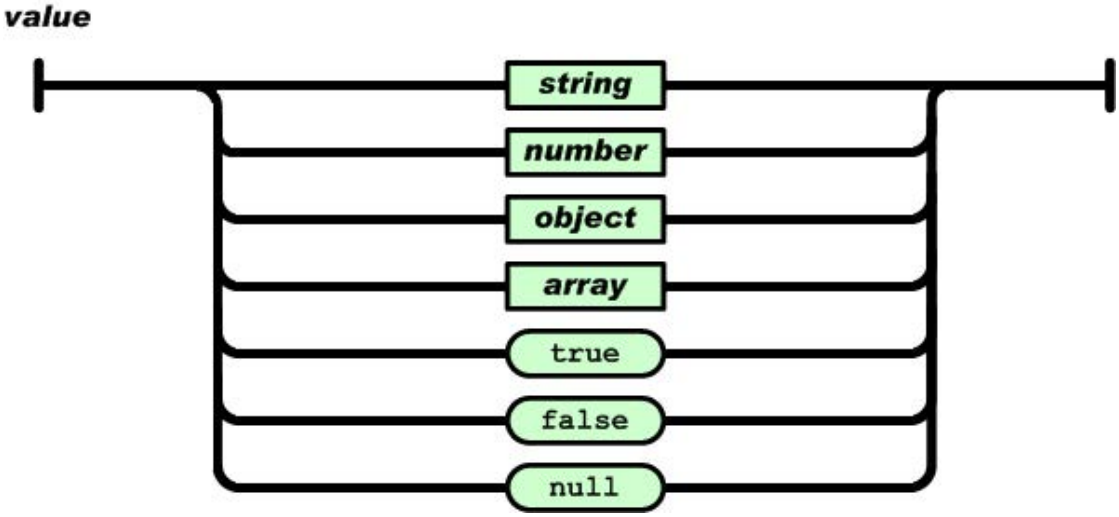

Figure 1.3: Data types

```
                "GlossSeeAlso": ["GML", "XML"]
              },
              "GlossSee": "markup"
            }
          }
        }
      }
    }
```

The environment responsible for processing these data formats is the web browser, which utilizes specialized object models to interface with the user and the document.

1.1 JSON serialization, field order and unsupported values

Let us consider the following code:

```
function print_name() {console.log(this.name)}

let example_obj = {
        "name": "Davide",
        "surname": "Loconte",
        "id": 590095,

        // Nested object
        "address": {
                "city": "Bari",
                "CAP": 70125
        },
```

```
        /*
                If we use an array the order
                of the values is preserved.
        */
        "subjects": ["web", "cs", "ai"],

        // We can associate a function reference to a key
        "print_name": print_name
}

console.log(JSON.stringify(example_obj));
```

In this code we define a function and a JavaScript object and then we print a stringified version of the latter. The output is as follows:

{”name”:”Davide”,”surname”:”Loconte”,”id”:590095, ”address”:”city”:”Bari”,”CAP”:70125, ”subjects”:[”web”,”cs”,”ai”]}

We can notice that the key `"print_name"` is absent because its value is a function reference. JSON was made for data interchange, not code interchange. JSON supports only serializable data meaning that functions, `undefined` and symbols are ignored during the stringification process. Not to mention the fact that, even if the function body was transferred to another machine, it wouldn't be granted that the receiving application can execute the received code because the receiving application might be written in a different language.

As for the order of the fields in the serialized string, theoretically it can change according to the implementation of the serializer and deserializer because JSON is made to define objects as an unordered set of key-value pairs. For the arrays, instead, the order of the values is preserved.

In modern JavaScript engine implementations the order of the fields is predictable because it follows the following rules:

- integer-like keys (strings representing integers) → ascending order
- string keys → insertion order order

1.2 JSON Schema

JSON Schema is a vocabulary that allows defining the structure, constraints, and validation rules for JSON data. It is used to describe the expected format of JSON documents and to ensure that the data exchanged between systems is consistent and valid.

1.2.1 Purpose and Motivation

In many web applications, JSON is used as the primary format for data exchange. However, JSON by itself does not enforce any structure or constraints. JSON Schema addresses this limitation by:

- defining the expected structure of JSON data;

- validating data before processing it;
- improving interoperability between systems;
- providing documentation for APIs and data models.

1.2.2 Basic Structure

A JSON Schema is itself a JSON document that describes the structure of another JSON object. A simple example is shown below:

```
{
  "type": "object",
  "properties": {
        "name": { "type": "string" },
        "age":  { "type": "number" }
  },
  "required": ["name"]
}
```

This schema specifies that:

- the data must be an object;
- it may contain `name` and `age` properties;
- the `name` property is mandatory;
- `name` must be a string, while `age` must be a number.

1.2.3 Common Keywords

JSON Schema provides several keywords to define constraints:

- `type`: specifies the data type (e.g., `string`, `number`, `object`, `array`, `boolean`, `null`);
- `properties`: defines the properties of an object and their respective schemas;
- `required`: lists the mandatory properties;
- `items`: defines the schema for elements in an array;
- `enum`: restricts values to a predefined set;
- `minimum`, `maximum`: define numeric constraints;
- `minLength`, `maxLength`: define constraints for string length;
- `pattern`: specifies a regular expression that a string must match.

1.2.4 Example with Validation Constraints

```
{
  "type": "object",
  "properties": {
        "username": {
          "type": "string",
          "minLength": 3
        },
        "password": {
          "type": "string",
          "minLength": 8
        },
        "age": {
          "type": "number",
          "minimum": 18
        }
  },
  "required": ["username", "password"]
}
```

This schema enforces constraints on the structure and content of the JSON data, ensuring that only valid inputs are accepted.

1.2.5 Advantages

- Improves data reliability by enforcing constraints;
- Reduces runtime errors;
- Provides a formal and machine-readable specification of data formats;
- Facilitates integration between heterogeneous systems.

In summary, JSON Schema is a powerful tool for defining and validating the structure of JSON data, making it essential in modern web development and API design.

1.3 JSON vs XML: Structure, Validation, and Performance

JSON (JavaScript Object Notation) and XML (eXtensible Markup Language) are two widely used formats for data representation and exchange. While both serve similar purposes, they differ significantly in syntax, structure, validation mechanisms, and performance.

1.3.1 Structural Differences

- **Syntax:** JSON uses a lightweight, key-value pair syntax, while XML relies on a tag-based structure with opening and closing tags.
- **Readability:** JSON is generally more concise and easier to read, whereas XML is more verbose due to repeated tags.

- **Data Representation:** JSON directly supports data types such as numbers, strings, arrays, and booleans. XML represents all data as text and requires additional parsing to interpret types.
- **Parsing:** JSON is natively supported in JavaScript and can be parsed quickly. XML requires a parser (e.g., DOM or SAX), making it more complex to process.

1.3.2 Example and Space Comparison

Consider the following data describing a user:

```
{
  "user": {
        "name": "Alice",
        "age": 25,
        "email": "alice@example.com"
  }
}
```

Listing 1.1: JSON representation

```
<user>
  <name>Alice</name>
  <age>25</age>
  <email>alice@example.com</email>
</user>
```

Listing 1.2: XML representation

The XML version is more verbose because each field requires both opening and closing tags. As a result:

- XML typically occupies **more disk space** than JSON;
- Larger size implies **higher bandwidth usage**;
- This leads to **longer data transfer times**, especially in networked applications such as web services.

For this reason, JSON is generally preferred in modern web applications where efficiency and speed are critical.

1.3.3 Validation Mechanisms

Both JSON and XML support validation, but they use different approaches.

XML Validation XML validation is traditionally performed using:

- **DTD (Document Type Definition)**;
- **XML Schema (XSD)**.

These mechanisms define the allowed structure, elements, attributes, and data types. XML validation is highly expressive and supports complex constraints, namespaces, and strict typing.

JSON Validation JSON validation is typically performed using **JSON Schema**, which defines:

- required properties;
- data types;
- constraints (e.g., ranges, string length, patterns).

Compared to XML Schema:

- JSON Schema is generally **simpler and more lightweight**;
- it is easier to read and integrate with modern web technologies;
- however, it may be less expressive than XML Schema in very complex scenarios.

1.3.4 Summary of Differences

- JSON is more **compact**, while XML is more **verbose**;
- JSON is easier to **parse** and more suitable for web applications;
- XML provides more **advanced validation** and extensibility features;
- JSON leads to better **performance** in terms of storage, bandwidth, and transmission time.

In conclusion, JSON is generally preferred for lightweight data exchange and web APIs, while XML is still used in contexts where strict validation, document structure, and extensibility are required.

2 Web API

The Web API in JavaScript provides a set of interfaces that allow interaction with the browser environment, enabling dynamic web content.

2.1 The Browser Environment: BOM and DOM Architectures

The browser operates on a dual-model architecture that serves as the foundation for web intelligence. This architecture distinguishes between the **Browser Object Model (BOM)**, representing the host environment, and the **Document Object Model (DOM)**, representing the specific content.

2.1.1 The Browser Object Model (BOM)

The BOM is centered on the `window` object, the global entity in the browser environment. It controls features beyond the document content, such as dialogs (`alert`, `confirm`), window management (`close`, `open`), event handlers, and navigation (`history`). The `window.location` property is critical for URL management and provides the following distinct properties:

- **hash:** The fragment identifier within the URL.
- **hostname:** The server's domain name or IP address.
- **port:** The communication port number (e.g., 80 or 443).
- **pathname:** The path to the specific resource on the server.
- **protocol:** The communication protocol used (e.g., HTTP or HTTPS).
- **search:** The query string containing parameters.

The history read-only property returns a reference to the History object, which provides an interface for consulting and manipulating the browser session history (pages visited in the tab or frame that the current page is loaded in). Using this interface it is possible to get the length of the tab history (`history.length`) and navigate through it using the following functions:

- **back():** This asynchronous method goes to the previous page in session history, the same action as when the user clicks the browser's **Back** button. Equivalent to `history.go(-1)`.
- **forward():** This asynchronous method goes to the next page in session history, the same action as when the user clicks the browser's **Forward** button; this is equivalent to `history.go(1)`. Calling this method to go forward beyond the most recent page in the session history has no effect and doesn't raise an exception.
- **go(int):** Asynchronously loads a page from the session history, identified by its relative location to the current page, for example -1 for the previous page or 1 for the next page. If you specify an out-of-bounds value (for instance, specifying -1 when there are no previously-visited pages in the session history), this method silently has no effect. Calling `go()` without parameters or a value of 0 reloads the current page.

Example of BOM usage

In the following code we will expose two common usages of the BOM.

```
<html>
  <head>
        <script>
        alert(
        `This is an alert. Use it to inform the
        user of something important.`
        );
        </script>
  </head>

  <body>
        <h1 id="label">Hello world!</h1>

        <script>
        let element = document.getElementById("label");
        confirm(
        `This is a confirm. Use it to ask the user for
        confirmations and permissions.\n For example:
        Will you remember that scripts at the bottom of the body
        can access elements in the page even if it hasn't
        been shown yet? The value of the text you will see in
        the page is: ${element.innerHTML}`
        );
        </script>
  </body>
</html>
```

The first common usage of the BOM is the `alert` command. This command generates a text box containing a message the user can acknowledge. Use it to inform the user of something important.

The second common usage is the `confirm` command. This command generates a text box with two buttons that make the user decide whether to confirm or refuse an operation. Unlike the `alert` command which returns `undefined`, the `confirm` command can return the following boolean values:

- Ok → `true`
- Cancel → `false`

Another important thing to notice about the script at the end is the difference between adding an element to the DOM and rendering (painting) it on the screen. An element is added to the DOM as soon as it is parsed, while it is rendered (painted) on the screen later in the rendering process.

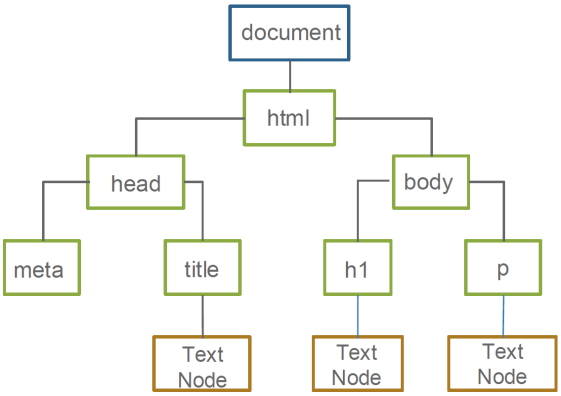

Figure 2.1: Document Object Model of the simple HTML document

2.1.2 The Document Object Model (DOM)

The DOM is the logical tree structure of an HTML document. It defines the hierarchical relationship between elements and provides the `document` variable as the primary entry point for JavaScript interaction. By treating the document as a tree of nodes, JavaScript can dynamically manipulate styles, attributes, and content via specific methods because each element of an HTML document implements the DOM element interface.

Here is an example of a simple HTML document and its relative DOM tree (figure 2.1).

```
<!DOCTYPE html>
<html>
        <head>
                <meta charset="UTF-8">
                <title>Example Page</title>
        </head>

        <body>
                <h1>DOM tree</h1>
                <p>Hello!</p>
        </body>
</html>
```

2.2 The Document Object: Attributes and Methods

The **document** object is a JavaScript object that provides the access to all elements of an HTML document. When the HTML document is loaded in the web browser, it creates a **document** object. It is the root of the HTML document.

The **document** object contains the various properties and methods you can use to get details about HTML elements and customize them.

The JavaScript **document** object is a property of the `window` object. It can be accessed using `window.document` syntax. It can also be accessed without prefixing `window` object.

2.2.1 Document Attributes

The `document` object provides several attributes that allow access to collections of elements within an HTML page. These attributes often return arrays (or array-like structures) containing elements in the order they appear in the document.

Some common attributes include:

- `document.embeds`: contains all embedded objects in the document;
- `document.forms`: contains all the forms defined in the page;
- `document.images`: contains all the images present in the document;
- `document.links`: contains all the hyperlinks.

Each of these collections can be accessed either by index or by identifier. For example:

```
document.forms[0];            // access by position
document.forms["FirstForm"];  // access by id
```

Another important attribute is `document.body`, which represents the body of the HTML document. It can be manipulated like any other DOM node through the standard element interface.

Additional useful attributes include:

- `document.cookie`: allows reading and modifying cookies;
- `document.lastModified`: returns the last modification date of the document;
- `document.referrer`: indicates the URL of the referring page;
- `document.title`: represents the title of the document;
- `document.URL`: contains the full URL of the document.

2.2.2 Document Methods

The `document` object also provides several methods for selecting and creating elements within the DOM.

Selecting Elements

- `getElementById("identifier")`: selects an element using its unique identifier.
- `querySelector("cssSelector")`: returns the first element that matches a given CSS selector.
- `querySelectorAll("cssSelector")`: returns all elements matching the CSS selector as a list of nodes.

Example:

```
const list = document.querySelector(".list");
```

Creating Elements

- `document.createElement("tag")`: creates a new element that can later be inserted into the DOM using standard node manipulation methods.

These methods provide the foundation for dynamically interacting with and modifying the structure of a web page.

2.2.3 Event Handlers

User interaction is mediated by event handlers that trigger functions in response to specific triggers:

Event Type	HTML Attribute	Trigger Description
Blur	`onBlur`	Loss of focus of an element. Blur is the opposite of focus.
Change	`onChange`	Modification of a text field, area, or selection list value.
Click	`onClick`	User activation of a specific page element.
Focus	`onFocus`	Highlighting or selection of a form element for input.
Load	`onLoad`	Successful completion of document or resource loading.
MouseOver	`onMouseOver`	Pointer enters the bounding box of an element.
MouseOut	`onMouseOut`	Pointer exits the bounding box of an element.
Select	`onSelect`	Selection of text within a form input or text area.
Submit	`onSubmit`	Submission of a form to the server.
Unload	`onUnload`	The current document is closed or replaced by a new one.

For JavaScript to interact with these models effectively, the timing of script execution must be strictly managed to avoid blocking the document parser.

2.2.4 Example of DOM manipulation and event handling

The following example shows how to manipulate the DOM and handle events.

```
<!DOCTYPE html>
<html>
  <body>
          <h1 id="main_text">Hello world!</h1>

          <script>
                  let div = document.createElement("div");
```

```
                div.append("Text added later");

                document.body.append(div);

                /*
                We can add an onclick function to an element
                by adding the 'onclick' attribute.
                */
                div.setAttribute('onclick',
                "alert(\'This functionality was implemented by " +
                "adding the onclick attribute.\')");

                // Alternatively, we can also add an event listener
                let main_text = document.getElementById("main_text");
                main_text.addEventListener("click", function(){
                      alert("This functionality was implemented by " +
                      "adding an event listener.");
                });
        </script>
  </body>
</html>
```

In this example, the body initially contains only a `Hello world!` message with an id that allows us to reference it later.

The script then creates a new `div` element and inserts text into it using the `append` method, which creates a text node. The element is then added to the document by appending it to the `body`.

After that, we define event handlers for user interactions. For the dynamically created `div`, the click handler is assigned by setting the `onclick` attribute, although this approach is generally discouraged. For the main text, the event is handled using the `addEventListener` method, which is the preferred approach.

2.3 The Element Interface: Attributes and Methods

Element is the most general base class from which all element objects (i.e., objects that represent elements) in a **Document** inherit. It only has methods and properties common to all kinds of elements. More specific classes inherit from **Element**.

For example, the **HTMLElement** interface is the base interface for HTML elements. Similarly, the **SVGElement** interface is the basis for all SVG elements, and the **MathMLElement** interface is the base interface for MathML elements.

2.3.1 Element attributes

The Element interface also exposes attributes that allow direct access to content and styling.

- `element.innerHTML`: allows reading or modifying the HTML content inside the element. This enables dynamic updates of the page structure.

  For example:

```
let div = document.getElementById("container");
div.innerHTML = "<p>Hello!</p>";
```

- `element.style`: returns an object representing the inline CSS styles of the element. Each CSS property is accessible as a corresponding property of this object, allowing dynamic changes to the element's appearance.

  For example:

```
el.style.marginTop = "20px";
```

Through these attributes and methods, developers can dynamically modify both the structure and the presentation of web pages.

2.3.2 Element methods

The Element interface provides several methods to navigate and manipulate the DOM subtree rooted at a specific element.

- `element.getElementsByTagName("tag")`: returns a collection of all descendant elements with the specified tag name within the subtree of the current element;

  For example:

```
/*
        Retrieve all the anchor elements
        nested in the 'el' element.
*/
anchorTags = el.getElementsByTagName("a");
```

- `element.getAttribute(attributeName)`: retrieves the value of a given attribute of the element;
- `element.setAttribute(name, value)`: sets or updates the value of a specified attribute;
- `element.appendChild(child)`: adds a new child node at the end of the list of children of the element;

  For example:

```
/*
        Create a paragraph and append it
        to the 'el' element.
*/
let p = document.createElement("p");
el.appendChild(p);
```

- `insertBefore(newNode, referenceNode)`: inserts a node before a specified child;
- `replaceChild(newNode, oldNode)`: replaces an existing child node;
- `removeChild(child)`: removes a child node.

3 Optimized Resource Execution: Script Loading Strategies

Traditional `<script>` tags are "parser-blocking." When the browser encounters a script, it halts DOM construction to download and execute it. Placing scripts in the `<head>` or middle of the document can prevent access to elements located further down the page and create rendering bottlenecks.

Modern architecture resolves this using `async` and `defer` attributes:

Feature	**`async`**	**`defer`**
Download Timing	Parallel with HTML parsing.	Parallel with HTML parsing.
Execution Timing	Immediately after download, interrupting parsing.	After parsing is complete, but before `DOMContentLoaded`.
Order Maintenance	No. Executes as soon as downloaded.	Yes. Follows the order of appearance in the document.

The `async` attribute is suitable for independent scripts (e.g., analytics) but presents "race condition" risks for dependent modules. `defer` is the preferred standard for scripts requiring a fully constructed DOM. Notably, scripts created dynamically via `document.createElement` default to `async`. To ensure these scripts execute in their original document order, the property must be explicitly set: `script.async = false`.

Figure 3.1 depicts the interaction between HTML parsing and script download and execution in the cases in which the script is handled normally or when the script has the `async` and `defer` attributes.

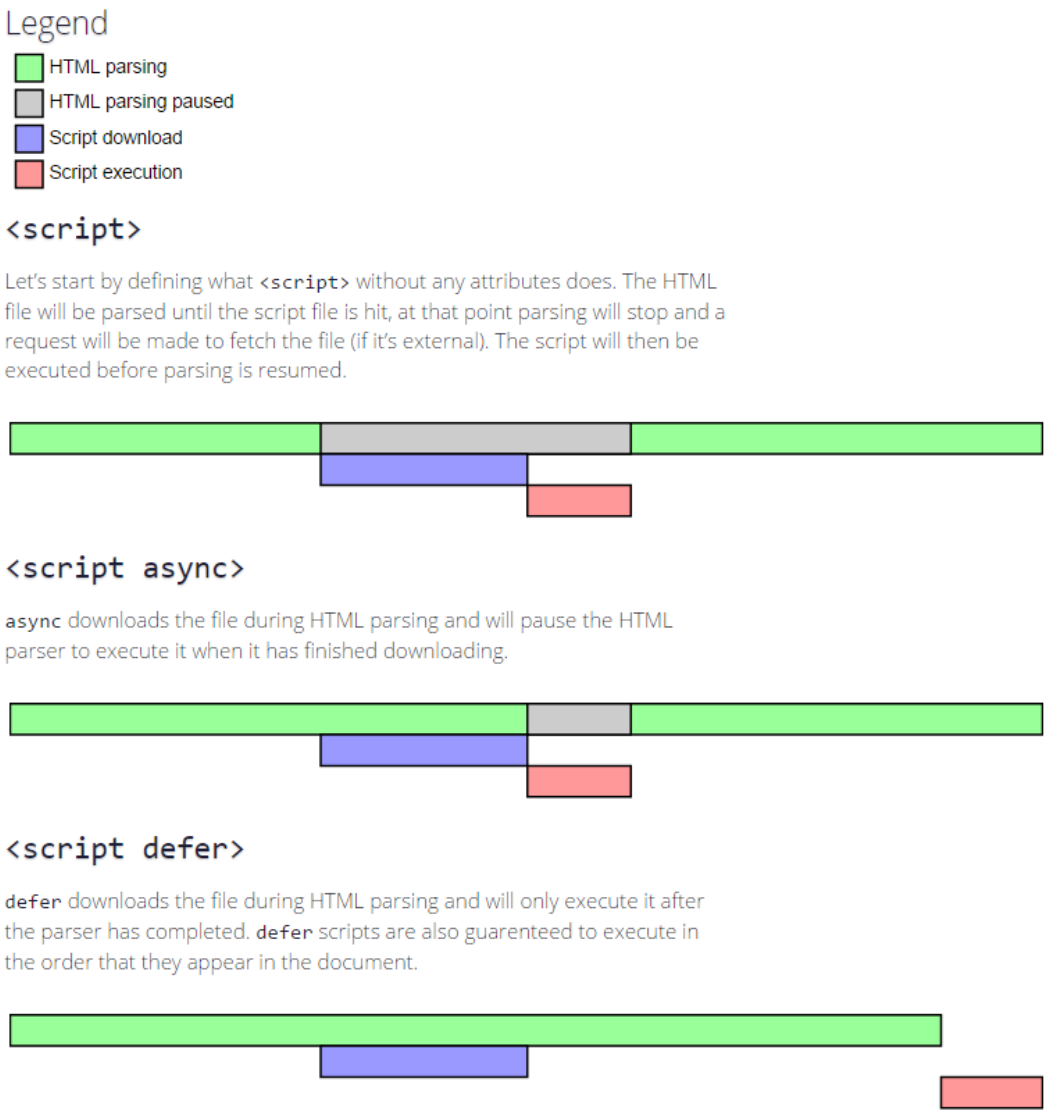

Figure 3.1: Interaction between the HTML parsing and the JavaScript script in the different handling modes

4 Other key Web APIs

4.1 Client-Side Persistence: Web Storage vs. Cookies

A shift from server-managed state to client-side persistence allows modern applications to retain user data without excessive network overhead. Web Storage APIs—`localStorage` and `sessionStorage`—provide persistent and temporary mechanisms governed by the Same-Origin Policy (matching protocol, domain, and port).

- **localStorage:** Data is stored on disk and persists across browser restarts until cleared.
- **sessionStorage:** Data is stored in RAM and is limited to the lifetime of the specific tab.

localStorage is generally used to store user preferences across sessions i.e. the website theme:

```
// In one session we can set the theme
localStorage.setItem("theme", "dark");

// In another session we can retrieve the theme
let theme = localStorage.getItem("theme"); // Dark
```

sessionStorage is used for temporary data like form inputs or login states within a single session:

```
// We can set the user value after a successful login
sessionStorage.setItem("user", "davide");

// We can retrieve it later
let user = sessionStorage.getItem("user"); // davide
```

4.1.1 Comparison: Web Storage vs. Cookies

Table 4.1 contains the comparison between cookies and local storage.

While Web Storage offers significantly higher capacity, cookies remain essential for authentication because the server can read session IDs directly during the HTTP request lifecycle.

4.2 Geolocation API

The Geolocation API further extends browser intelligence by providing location data. Precision varies by hardware: GPS-enabled mobile devices offer high accuracy, whereas desktop systems fall back to less precise IP-based lookups.

- **getCurrentPosition():** Retrieves the current location of the user

Feature	**Cookies**	**Web Storage (`localStorage` and `sessionStorage`)**
Size Limit	4 KB	**5 MB**
Data Type	Strings only	Serialized strings (typically JSON objects)
Server Transmission	Sent automatically with every HTTP request.	Never sent to the server automatically.
Security	Supports `HttpOnly` to mitigate XSS (Cross-Site Scripting, a form of malicious code injection attack).	Accessible via JS; vulnerable to XSS.
Scope	Can be shared across subdomains if configured.	Strictly limited to the specific domain.

Table 4.1: Comparison of cookies and local storage

- **watchPosition():** Tracks and updates the user's location as it changes

It is also possible to get the user latitude and longitude using the **getCurrentPosition()** API as follows:

```
navigator.geolocation.getCurrentPosition(
  (position) => {
          console.log("Latitude:", position.coords.latitude);
          console.log("Longitude:", position.coords.longitude);
  }
);
```

Regardless of precision, these APIs are restricted to secure contexts or remote websites to prevent unauthorized tracking from local files. In fact, using these APIs require explicit user permission and they are also governed by a Same-Origin Policy.

5 Asynchronous JavaScript (AJAX) and Network Interaction

AJAX is an architectural approach that enables the Single Page Application (SPA) model. Unlike the traditional Wikipedia-style model where every link triggers a full page refresh, AJAX uses an "AJAX Engine" (JavaScript) to fetch data in the background and update the DOM dynamically, as seen in applications like Instagram.

5.1 Synchronous requests vs asynchronous requests

In the traditional Wikipedia-style model requests are synchronous. This means that the **process cycle** is as follows (see figure 5.1):

1. The client sends the request to the server;
2. The client waits for the server to process and return the response;
3. The server sends the response to the client.

The AJAX model of a web application enables the client to send multiple **independent** requests to the server without reloading the page. In this model, the **process cycle** is different and the client interacts with the server through the AJAX engine which acts an intermediary (see figure 5.2) and the user is unaware of the underlying process.

On the one hand, AJAX has great advantages like:

- Compatibility with the most popular browsers;
- It makes web pages more interactive and usable to the eyes of the users;
- Makes serving resources more efficient: the server does not need to transfer the whole web page for each interaction, it only needs to transfer the necessary data.

On the other hand, advanced AJAX implementations introduce specific non-functional challenges regarding state management and system robustness because, as highlighted in the figure, there isn't a strict order among the operations in the process cycle. Notice that right before the first server transmission to the AJAX engine, the latter displays something to the user. It may seem like the AJAX engine can show data to the user before receiving it but it can't because it's physically impossible. Instead, the information you see displayed to the user is the response of a previous request to the server.

The following subsections expose how to address these issues.

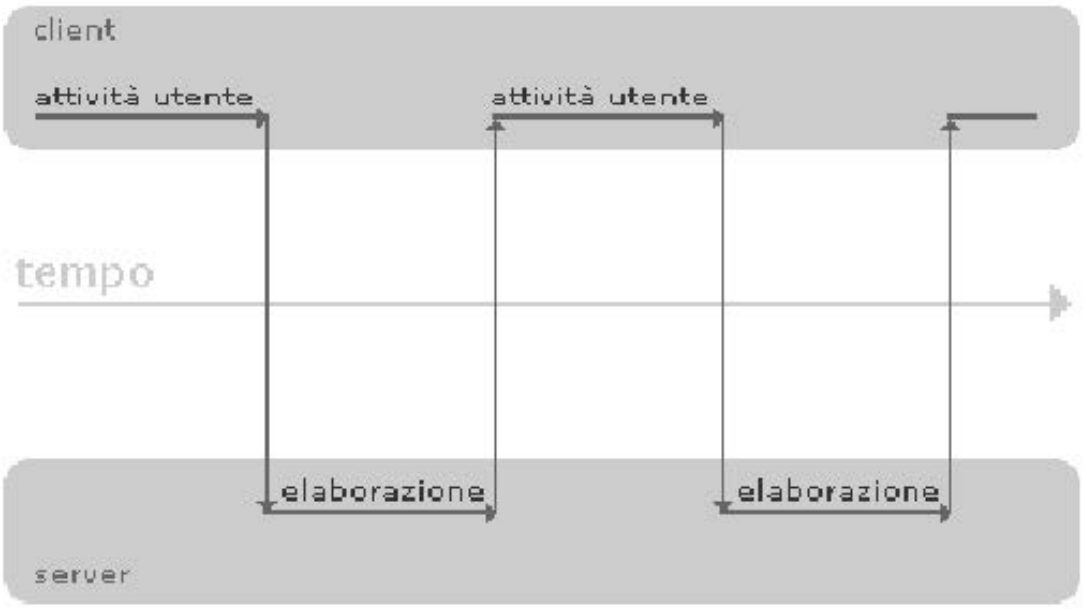

Figure 5.1: Synchronous request

5.1.1 Race Conditions in Search Inputs

A classic architectural bug occurs when a user types into a live-search bar. If a user types "cat" quickly, the browser sends requests for "c", "ca", and "cat". Because **independent** asynchronous responses do not guarantee order, the response for the first letter ("c") might arrive after the response for the full word ("cat"), causing the UI to display the wrong results. Solutions include:

1. **Debouncing:** Delaying the request until the user pauses typing.
2. **Request Cancellation:** Using `AbortController` to cancel previous unfinished requests.
3. **Versioning:** Tagging requests with IDs and ignoring any response that is not the most recent.

These technique are used by high quality search bars like the one by Google to make sure that the user only receives the right responses.

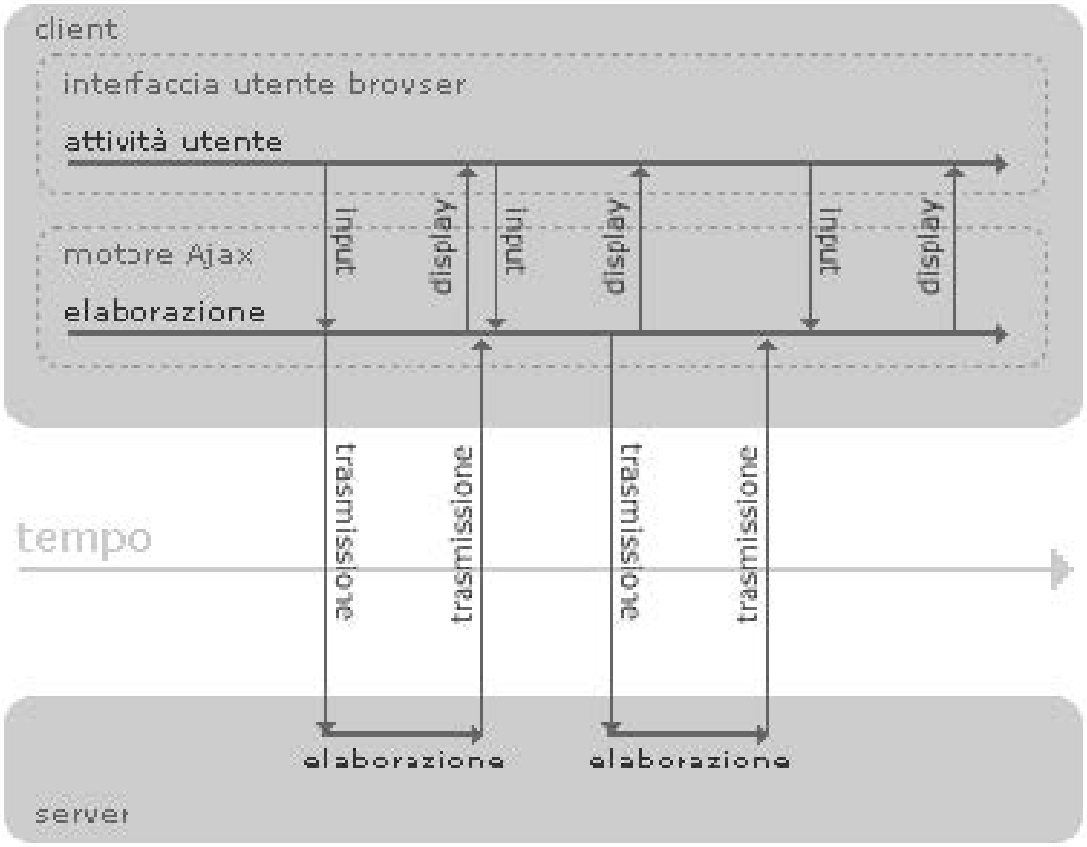

Figure 5.2: Asynchronous request

5.1.2 SEO and User Experience

AJAX-heavy sites break the traditional "one page = one URL" model. This interferes with **Search Engine Indexing**, as crawlers primarily read initial HTML, and breaks **Bookmark/History Management**. If a user clicks the "Back" button, the browser might navigate away from the site entirely rather than reverting an AJAX-driven state change.

Standard Solutions:

- **History API:** The industry standard for programmatically updating the URL and browser history without a page refresh, ensuring that "Back/Forward" navigation and bookmarks work as expected.
- **Server-Side Rendering (SSR):** Executing JavaScript on the server to provide a complete HTML document to search engine crawlers.

5.2 AJAX endpoints as attack surfaces and security considerations

AJAX endpoints represent a fundamental component of modern web applications, enabling asynchronous communication between client and server. However, from a security perspective, they also constitute critical access points that can be exploited by malicious users.

5.2.1 AJAX endpoints as vulnerability points

Every AJAX request targets a server-side endpoint that processes input and returns data. These endpoints are often exposed over HTTP and can be invoked programmatically, making them attractive targets for attackers.

Malicious users may exploit AJAX endpoints in several ways:

- **Denial of Service (DoS):** by issuing a large number of asynchronous requests, attackers can overwhelm the server, exhausting computational or network resources.
- **Unauthorized Data Access:** if endpoints lack proper authentication or authorization checks, attackers may retrieve sensitive information.
- **Injection Attacks:** improperly validated input sent via AJAX can lead to SQL injection, command injection, or other forms of exploitation.
- **Automation of Attacks:** since AJAX calls are easily reproducible, attackers can script repeated requests to systematically probe the system.

These characteristics make AJAX endpoints a significant part of the application's attack surface.

5.2.2 Same-Origin Policy and CORS

A first line of defense in web security is the *Same-Origin Policy*, which prevents browsers from making requests to a different origin (defined by protocol, domain, and port) unless explicitly allowed.

To enable controlled cross-origin interactions, servers can adopt *Cross-Origin Resource Sharing (CORS)*. This mechanism relies on specific HTTP headers:

- `Access-Control-Allow-Origin`: defines which origins are allowed to access the resource.
- `Access-Control-Allow-Methods`: specifies permitted HTTP methods (e.g., GET, POST).
- `Access-Control-Allow-Headers`: lists allowed request headers.
- `Access-Control-Allow-Credentials`: indicates whether cookies can be included.

Additionally, browsers distinguish between *simple requests* (e.g., basic GET requests) and *preflight requests*, where an initial `OPTIONS` request is sent to verify permissions before executing more complex operations.

While CORS provides flexibility, misconfiguration (e.g., allowing all origins with `*`) can expose endpoints to unauthorized access.

5.2.3 Defense Mechanisms

To mitigate risks associated with AJAX endpoints, several defensive strategies should be adopted:

- **Authentication and Authorization:** ensure that every endpoint verifies user identity and permissions;
- **Rate Limiting:** limit the number of requests per client to prevent DoS attacks;
- **Input Validation and Sanitization:** validate all incoming data to prevent injection attacks;
- **Secure CORS Configuration:** restrict allowed origins and methods to only trusted ones;
- **Use of Tokens (e.g., CSRF Tokens):** protect endpoints from cross-site request forgery attacks.

5.2.4 Example of a Complete CORS Header Configuration

The following example shows a possible HTTP response header configuration that includes all the main CORS directives discussed previously:

```
HTTP/1.1 200 OK
Content-Type: application/json

Access-Control-Allow-Origin: https://example.com
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Credentials: true
```

In this configuration:

- `Access-Control-Allow-Origin` restricts access to a specific trusted domain;

- `Access-Control-Allow-Methods` explicitly lists the allowed HTTP methods;
- `Access-Control-Allow-Headers` defines which custom headers can be used in requests;
- `Access-Control-Allow-Credentials` enables the inclusion of cookies and authentication data.

For completeness, a preflight response (to an `OPTIONS` request) may include the same headers:

```
HTTP/1.1 204 No Content

Access-Control-Allow-Origin: https://example.com
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Credentials: true
```

This configuration ensures controlled cross-origin access while maintaining a secure policy by limiting interactions to trusted clients.

In summary, AJAX endpoints significantly enhance interactivity but also introduce security risks. The Same-Origin Policy and CORS mechanisms help regulate cross-origin access, but they must be carefully configured. A secure design requires combining browser-level protections with robust server-side controls to reduce the exposure of these endpoints to malicious exploitation.

5.3 The `XMLHttpRequest` (XHR) and the Fetch API

The `XMLHttpRequest` (XHR) object is the traditional mechanism used to perform AJAX requests. It provides fine-grained control over HTTP communication, but requires a more verbose and event-driven programming style.

5.3.1 XHR Lifecycle and readyState property

An XHR request evolves through different states, tracked by the `readyState` property:

- **0 (UNSENT):** object created, but `open()` not yet called;
- **1 (OPENED):** `open()` has been called;
- **2 (HEADERS_RECEIVED):** `send()` has been called, response headers are available;
- **3 (LOADING):** response data is being received;
- **4 (DONE):** request completed and response fully available.

The `onreadystatechange` callback is triggered whenever the state changes and is commonly used to handle the response.

XHR request lifecycle example

This is an example showing how the state changes during the lifecycle of an XHR request.

```
const xhr = new XMLHttpRequest();
console.log("UNSENT", xhr.readyState); //readyState 0

xhr.open("GET", "/api", true);
console.log("OPENED", xhr.readyState); //readyState 1

xhr.onprogress = () => {
        console.log("LOADING", xhr.readyState); //readyState 3
};

xhr.onload = () => {
        console.log("DONE", xhr.readyState); //readyState 4
};

xhr.send(null);
```

5.3.2 XHR Methods and Execution Flow

XHRs have the following methods:

1. `open(method, url [, async] [, user] [, password])`: this method initializes the request and has the following arguments:
   - `method`: The HTTP method (GET, POST, PUT, DELETE);
   - `url`: The URL of the resource to send the request to;
   - `async`: Optional boolean parameter specifying if the request is synchronous (false) or asynchronous (true, default value);
   - `user` and `password`: Optional credentials for authentication purposes. Their default value is `null`.
2. `send([data])`: sends the request to the server. The `data` parameter is optional and is used for POST and PUT requests. Its default value is `null`.
3. `setRequestHeader(header, value)`: Sets the value of an HTTP request header, for example to specify the content type of data to be sent to the server. It must be called after `open()` and before `send()`. `setRequestHeader(header, value)` does not accept arrays as input, therefore it must be called once for each header. If the same header is set multiple times, its values are typically concatenated into a single header separated by commas, rather than being overwritten.

   For example:

   ```
   xhr.setRequestHeader("Content-Type", "application/json");
   xhr.setRequestHeader("X-Auth", "123");
   xhr.setRequestHeader("X-Auth", "456");
   ```

In this case the header becomes:

```
Content-Type: application/json
X-Auth: 123, 456
```

4. `getAllResponseHeaders()`: Returns a string containing all the response headers separated by a CRLF character;
5. `getResponseHeader(headerName)`: Returns a string containing the value of the requested header specified using the `headerName` parameter.

The typical workflow of an XHR request involves three main steps:

1. **`open(method, url, async)`**;
2. Optionally one or more **`setRequestHeader()`**;
3. **`send(data)`**.

Why is `open()` required before `send()`?

The `open()` method does **not** establish a network connection or perform a handshake with the server. Instead, it:

- configures the request by specifying the HTTP method, target URL, and whether it is asynchronous;
- transitions the object from state 0 (UNSENT) to state 1 (OPENED);
- prepares the internal request structure so that headers and callbacks can be correctly attached.

The actual network communication starts only when `send()` is invoked. Therefore:

- there is **no preliminary handshake** triggered by `open()`;
- `open()` is a configuration step, not a connectivity check;
- event handlers such as `onreadystatechange` should be defined before `send()` to avoid missing state transitions.

5.3.3 XHR properties

In addition to the `readyState` property, which tracks the lifecycle of a request, the `XMLHttpRequest` object provides several other useful properties that allow developers to inspect the response and control how data is handled.

- `status`: represents the HTTP status code returned by the server (e.g., 200 for success, 404 for not found, 500 for server error). It is commonly used to determine whether a request has been successfully completed;
- `statusText`: a textual description of the HTTP status code (e.g., “OK”, “Not Found”). It is mainly useful for debugging and logging purposes;
- `responseType`: specifies the expected format of the response. It must be set after calling `open()` and before calling `send()`. Possible values include:

  - `"text"`: plain text (default);
  - `"json"`: JSON data automatically parsed into a JavaScript object;
  - `"document"`: HTML or XML document;
  - `"blob"` or `"arraybuffer"`: binary data.

- `response`: contains the response data in a format consistent with the value of `responseType`. For example, if `responseType = "json"`, this property contains a JavaScript object;

- `responseText`: contains the response as a plain text string. This property is typically used when `responseType` is `"text"` or left unspecified;

- `responseXML`: contains the response parsed as an XML document. It is valid only when the response is XML or when `responseType` is set to `"document"`;

- `onreadystatechange`: an event handler (callback function) that is invoked every time the `readyState` property changes.

  This property is fundamental for handling asynchronous requests, as it allows the developer to monitor the progression of the request and execute code when the operation is completed.

  A common pattern is to check when the request reaches the `DONE` state (i.e., `readyState === 4` or, alternatively, `xhr.readyState === XMLHttpRequest.DONE` as XHR ready states are enumerated) and then verify the HTTP status:

```
xhr.onreadystatechange = function() {
  if(xhr.readyState === XMLHttpRequest.DONE) {
          const status = xhr.status;
          if (status === 0 || (status >= 200 && status < 400)) {
                  // the request has been completed successfully
                  console.log(xhr.responseText);
          } else {
                  // there is some error with the request!
                  console.error(`${xhr.status}: ${xhr.statusText}`)
          }
  }
};
```

  It is important to define `onreadystatechange` before calling `send()`, otherwise some state transitions might not be captured.

  Modern alternatives to this handler include `onload` (triggered when the request completes successfully) and `onerror` (triggered in case of network errors).

These properties allow developers to handle different types of responses and to build more robust and flexible AJAX-based applications.

5.3.4 Example of XMLHttpRequest

This is a full example of XHR request to the website https://jsonplaceholder.typicode.com/ which provides a free fake and reliable API for testing and prototyping.

The reader is encouraged to copy and paste the script in a browser console to check its output for themselves and memorize the example because XHR requests are recurrent when programming JS front-end scripts.

```
const xhr = new XMLHttpRequest();

// We have to write this function first and then send the request.
xhr.onreadystatechange = function (response) {
  /*
        Make sure to check the readyState an remember
        that the ready states are enumeratedas follows:
          0: UNSENT
          1: OPENED
          2: HEADERS_RECEIVED
          3: LOADING
          4: DONE
  */
  if (xhr.readyState === XMLHttpRequest.HEADERS_RECEIVED){
  console.log("The headers have been received");
  console.log("XHR ready state: " + xhr.readyState);
        console.log(xhr);
  }

  if (xhr.readyState === XMLHttpRequest.LOADING){
        console.log("The response is being loaded");
        console.log("XHR ready state: " + xhr.readyState);
        console.log(xhr);
  }

  if (xhr.readyState === XMLHttpRequest.DONE){
        console.log("Calling on ready state change");
        console.log("XHR ready state: " + xhr.readyState);
        console.log(xhr);
        console.log(JSON.parse(xhr.response));
  }
}

// Now the request is unsent
console.log("The request is unsent");
console.log("XHR ready state: " + xhr.readyState);
console.log(xhr);

xhr.open('GET', 'https://jsonplaceholder.typicode.com/todos/1');

// Now the request is opened
```

```
console.log("The request is opened");
console.log("XHR ready state: " + xhr.readyState);
console.log(xhr);

xhr.send();
```

5.3.5 Fetch API: A Modern Approach

The Fetch API provides a modern alternative to XHR, based on Promises rather than event callbacks.

Key characteristics:

- **Promise-based model:** asynchronous flows are handled using `then()` and `catch()`, improving readability. The following snippet illustrates an example of failing request.

  ```
  fetch('https://api.example.com/data')
  .then(response => {
    if (!response.ok) throw new Error('Request failed');
    return response.json(); // Parse JSON response
  })
  .then(data => console.log(data))
  .catch(error => console.error('Error:', error));
  ```

  This request fails because the URL `'https://api.example.com/data'` does not exist so, `response.ok = false`, the error object is thrown and is caught in the last line of the snippet;

- **Cleaner syntax:** avoids manual state tracking (`readyState`);
- **Built-in JSON handling:** methods like `response.json()` simplify parsing;
- **Flexible configuration:** supports custom HTTP methods, headers, and request bodies. For example, a POST request is as follows:

  ```
  fetch('https://api.example.com/data', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ key: 'value' })
  });
  ```

5.3.6 Example of request using the fetch API

Now, let us rewrite the same example as before but using the fetch API.

```
fetch('https://jsonplaceholder.typicode.com/todos/1')
.then((response) => {
  return response.json();
})
```

```
.then(response => {
  console.log(response);
})
  .catch((response) => {
        console.log("There was an error");
});
```

5.3.7 XHR vs Fetch: Key Differences

- **Programming model:** XHR is event-driven, Fetch is promise-based;
- **Complexity:** XHR requires manual state management, whereas Fetch is more concise. XHR provides fine-grained control over the request lifecycle; however, this level of control is not required in most cases. For this reason, the Fetch API was introduced to abstract away explicit state management and simplify asynchronous programming;
- **Error handling:** Fetch uses promise rejection, while XHR relies on status checks and event handlers;
- **Modern usage:** Fetch is the preferred standard for new applications, while XHR is maintained mainly for backward compatibility.

In summary, XHR offers low-level control over HTTP requests, while Fetch provides a higher-level abstraction that simplifies asynchronous programming.